

Київський національний університет імені Тараса
Шевченка Факультет комп'ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

Лабораторна робота №2

Інструментальні засоби розробки програмного
забезпечення

Виконала студентка 2-го курсу
Групи ІПС-22
Чергінець Юлія Дмитрівна

Київ – 2025

Тема:

Документація коду (Doxxygen/JavaDoc/...) і автоматична публікація через GitHub Actions та GitHub Pages.

Мета:

Навчитися:

- документувати код стандартними коментарями (Doxxygen, JavaDoc тощо);
- автоматично генерувати HTML-документацію в CI/CD;
- публікувати її на GitHub Pages після кожного коміту.

Результат:

1. Посилання на репозиторій GitHub.

<https://github.com/raychiikk/lab1-instrumental>

2. Посилання на сторінку GitHub Pages із документацією.

<https://raychiikk.github.io/lab1-instrumental/docs/>

3. Скріншоти або опис основних кроків (коментарі, CI, сторінка).

Опис основних кроків

1. Документування коду (Коментарі JSDoc)

Першим кроком було додавання коментарів до існуючої кодової бази, щоб забезпечити можливість автоматичної генерації документації.

1. **Вибір інструменту:** Для проєкту на JavaScript було обрано JSDoc.
2. **Встановлення:** JSDoc було додано до проєкту як залежність для розробки (npm install -D jsdoc).
3. **Коментування:** До основних файлів логіки (src/utils/todoUtils.js та src/hooks/useTodos.js), а також до головного компонента (src/App.jsx), було додано JSDoc-блоки (`/** ... */`) перед кожною функцією та хуком.

- Використано ключові теги, як того вимагало завдання:
 - **@brief**: Для короткого опису функції.
 - **@param**: Для опису кожного аргументу, його типу та призначення.
 - **@return**: Для опису значення, що повертається.
 - **@throws**: Для опису помилок, які може викинути функція (наприклад, у `validateTodoText`).
 - **@example**: Для надання прикладу використання коду.
 - **@typedef**: Для визначення кастомних типів даних (наприклад, `Todo` та `UseTodosReturn`), що значно покращило читабельність документації для складних об'єктів.
- 4. **Конфігурація**: Створено файл `jsdoc.conf.json` у корені проєкту. Цей файл налаштовує JSDoc, вказуючи йому:
 - Вихідну папку для сканування (було обрано всю папку **src**, щоб включити всі хуки та компоненти).
 - Патерни файлів, які слід ігнорувати (наприклад, `*.test.js`, `node_modules`, `dist`).
 - **Фінальну папку для виводу** згенерованого HTML (встановлено на `dist/docs/`, щоб документація публікувалася разом із зібраним проєктом).
- 5. **Локальна перевірка**: За допомогою команди `npx jsdoc -c jsdoc.conf.json` було протестовано генерацію документації.
- 6. **Ігнорування**: Папка `dist` (яка тепер містить і документацію, і зібраний додаток) вже була додана до `.gitignore`.

2. Налаштування CI/CD (GitHub Actions)

Для автоматизації процесу збірки та публікації **і додатку, і документації**, було налаштовано єдиний воркфлоу (workflow).

1. **Створення воркфлоу**: У репозиторії було створено файл `.github/workflows/docs.yml`.
2. **Тригери**: Воркфлоу налаштовано на автоматичний запуск при кожному `push` у гілці `main` та `feature/docs-ci`.
3. **Задачі (Jobs)**: Воркфлоу складається з однієї об'єднаної задачі `build-and-deploy`, яка виконує повний цикл збірки та публікації:

1. Завантажує код репозиторію (actions/checkout).
 2. Встановлює середовище Node.js (actions/setup-node).
 3. Встановлює всі залежності проєкту (npm ci).
 4. **Збирає React-додаток** (npm run build). Цей крок створює папку dist з оптимізованим кодом додатку.
 5. **Генерує документацію** (npx jsdoc -c jsdoc.conf.json). Цей крок генерує HTML-файли документації і кладе їх у **вже існуючу** папку dist за шляхом dist/docs/.
 6. Запаковує **всю папку dist** (яка тепер містить і додаток, і документацію) як артефакт (actions/upload-pages-artifact).
 7. Використовує стандартний actions/deploy-pages для взяття артефакту та його публікації на GitHub Pages.
4. **Дозволи (Permissions):** Файлу воркфлоу було надано необхідні дозволи (pages: write, id-token: write) для права запису на GitHub Pages.

3. Публікація (GitHub Pages)

Фінальним кроком було налаштування самого репозиторію GitHub для прийому та відображення згенерованої сторінки.

1. **Налаштування джерела:** У налаштуваннях репозиторію (Settings > Pages) у розділі "Build and deployment" джерело (Source) було змінено з "Deploy from a branch" на **"GitHub Actions"**.
2. **Налаштування середовища (Environment):** Під час деплою з feature-гілки виникла помилка безпеки. Для її вирішення у Settings > Environments > github-pages було додано правило "Deployment branches", яке дозволило гілці feature/docs-сі виконувати деплой.
3. **Перевірка:** Після внесення цих налаштувань воркфлоу було перезапущено. Обидва процеси (збірка додатку та генерація доків) успішно завершилася.
4. **Результат:** У розділі Settings > Pages з'явилося активне посилання.
 - За посиланням <https://raychiikk.github.io/lab1-instrumental/> знаходиться сам React-додаток.
 - За вкладеним посиланням <https://raychiikk.github.io/lab1-instrumental/docs/> — згенерована JSDoc-документація.

4. Короткий висновок про набуті навички.

Висновок:

Під час виконання цієї лабораторної роботи я здобула практичні навички у:

- **Документуванні коду:** Навчилась застосовувати стандарт JSDoc для детального опису функцій, хуків, компонентів, їхніх параметрів (@param), значень, що повертаються (@return), та кастомних типів даних (@typedef).
- **Автоматизації (CI/CD):** Здобула вміння створювати "з нуля" воркфлоу (workflow) в GitHub Actions.
- **Публікації (Деплої):** Навчилась налаштовувати GitHub Pages для автоматичної публікації статичних веб-сайтів (документації) через CI/CD.
- **Інтеграції та вирішенні проблем:** Здобула досвід у поєднанні різних сервісів GitHub (Actions, Pages, Environments) та у вирішенні реальних проблем, пов'язаних з правами доступу (наприклад, налаштування Environments).

У результаті, я вмію налаштовувати повний цикл автоматичної доставки документації: від написання коментаря в коді до його появи на опублікованому сайті.